Tutorial 2 – Uploading data and some simple visualizations

If you completed tutorial 1, you should have Rstudio installed and working. In this tutorial we will go through the process of uploading data, rearranging the data and plotting some interesting visualizations. The goal is to get you interested in learning more, and to get you started on the right foot.

Uploading data into R-

As I mentioned before, R is an object-oriented language. Thus, if you want R to have a data set in its "brain", we need to create and object with this data set. There are two general ways to do this: 1) you have the data in some sort of format saved in your computer; 2) the data exists online. Let's go through each of these situations.

Use a data set saved into your computer. For example, you can create the following dataset using excel, google data, or any other software:

	A	B	C	D	E
1	plot	treatment	corn_height	bean_height	
2	A	Y	120	62	
3	A	Y	130	65	
4	A	Y	125	60	
5	A	Y	118	61	
6	A	N	97	61	
7	A	N	120	64	
8	A	N	102	59	
9	A	N	112	58	
10	B	Y	132	62	
11	B	Y	121	61	
12	B	Y	124	65	
13	B	Y	121	63	
14	B	N	100	63	
15	B	N	118	60	
16	B	N	102	58	
17	B	N	120	57	

Save this data as fertilizer.csv

It is important to choose the correct format. The basic R package reads csv files. These are files where each line have the values from each column separated by commes. csv = comma separated values. Most software allows you to save data in this format. Rstudio also upload excel files without problems, so files saved as data.xlx should be fine. But other format might require special packages to be installed in R to use it, so we will not cover here.

To upload or import into R, go to the top right window (the environment pane), and click on the environment tab. Right underneath you will find a button named "import dataset". Click it and select "From text (base)….". If you had an excel file, click "From excel". That would allow you to browse towards the correct file. Highlight fertilizer.csv and click open. If all went well you should now see a tab on top left appear saying fertilizer. More importantly, you will see in the console the commands:

```
> fertilizer = read.csv("~/Documents/fertilizer.csv")
>View(fertilizer)
```

What that means, is that importing a file into R is the equivalent of creating an object named fertilizer that contains the data set.

R studio did this automatically when you clicked “import Dataset” button. Which is really nice. However, R studio chose the name of the object for you, which may or may not be helpful. It usually uses the file name, which can be long and cumbersome sometimes….

I want to reassure you that once you see the commands, it is easy for you to change it and do it your way. You can go to the console pane, and click on the new line with the prompt >

you can now type the same thing but give the object a different name:

dataset= read.csv(“~/Documents/fertilizer.csv”)

That will create a new object called dataset with the same information as the object fertilizer.

Recycling your previous commands-

If you want to create 200 objects with the same information you can, but why would you? Well, if you want to do this, it might be handy to learn that if you are in a new prompt and you press the upwards arrow, it will scroll through all the commands you have used in the R session. In this case, it would be super handy because you can use the up arrow to get back to the line that name the object, edit the name of the object and press enter, and voila’ you created yet a new object with the same contents!

If you are using the script pane, you can just change the object name and run that line again!

The best way to check that you gave the right command, is to view the object you just created. You can do that by typing:

view(object_name)

or

view(dataset)

This will allow you to actually see what is inside the object and check that all the columns and lines are there, in the correct way.

Another nice way to check is all there is by typing

>str(fertilizer)

```
R will output:
     'data.frame':   16 obs. of  4 variables:
     $ plot       : chr  "A" "A" "A" "A" ...
     $ treatment : chr  "Y" "Y" "Y" "Y" ...
     $ corn_height: int  120 130 125 118 97 120 102 112 132 121 ...
     $ bean_height: int  62 65 60 61 61 64 59 58 62 61 ...
```

str() is the structure function, so what the command above says is: Provide the structure of the object fertilizer. That shows that fertilizer is a datframe with 4 variables, as expected; where the first two variable only have characters (chr) and the last two have integers or numbers (int).

Sometimes, when R seems to not be doing what you want is because your variable is thought as a character variable, because you entered NA or X for a missing value. A quick check of the structure would make this clear and you can fix it. It is also super helpful when you open a file that you do not know what is in it, as you will see later.

Visualize some data trends:

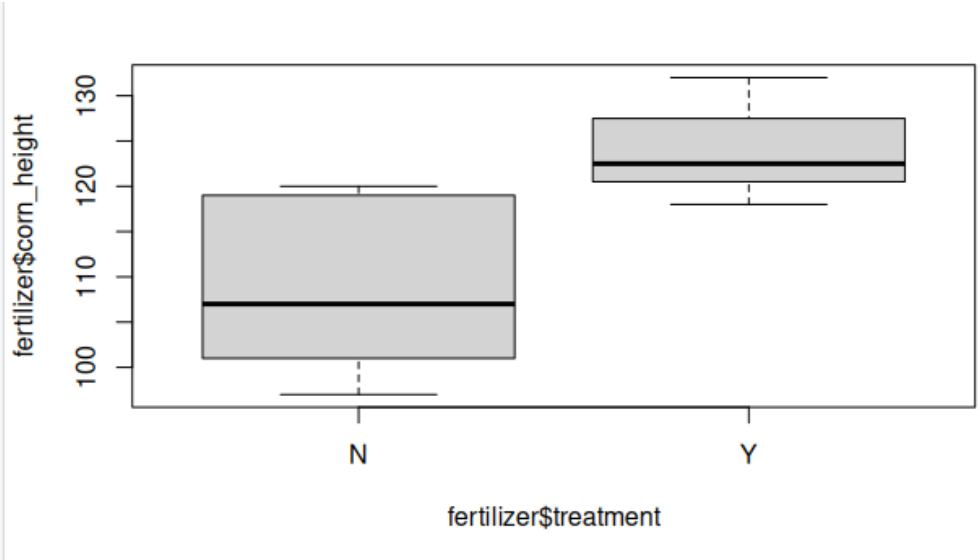
The dataset fertilizer.csv is made up, but it represents an experiment where two plots of lands were divided halfs: one that received fertilizer and the other that did not. Bean and corn plants were grown in these plots and their heights measured as an indicator how well these plants were growing with and without fertilizer in two plots. It makes sense then to visualize whether plants grown with fertilizer are taller than plants grown without it.

To do this, we want to plot a graph where the x axis has fertilizer or no fertilizer, and in the y axis we have the average height of the plants. A simple bar graph can be created in R using the following command:

```
boxplot(fertilizer$corn_height ~ fertilizer$fertilizer)
```

Here is the translation of this command: Do a boxplot of the variable corn_height that exists within the object fertilizer, using the variable treatment in the object fertilizer in the x axis)

and here is the output:



So it is possible to see quite quickly that corn plants in this experiment tend to to have taller plants when grown with the fertilizer. You need to run some statistics to say that are statistically significantly different. We will look into that later.

Challenge: Can you produce a similar graph to see if bean plants are taller with fertilizer? Answer you will be provided at the end of the document

R uses objects and functions:

Objects were introduced earlier on, and now is time to introduce functions. A function is how you tell R to make an action. The function will have a name and then a set of conditions. We already saw many functions! For example: str() , read.csv(), boxplot(). You notice that when I list the function there are always a set of parenthesis. If this is completely blank, it will run the default. But usually this is where we add the specifics. So when we say str(fertilizer), we are specifying that we want run the function on the object fertilizer. We can add quite a lot of detail within the parenthesis as you will see next. Once you master R you would be writing your own functions!